AP4A : Logiciel de simulation IoT

Cuney Alexandre

Guyenot-Cosio Amber



Table des matières

Introduction....2
Note sur l'IA....2
Conception....3
Structure du projet....3
Class et objets....3
Server....4
Sensor....5
Scheduler....7
Room....7
Data....8
Ipv4....8
ConfigLoader....9
Fonctionnement....9
Axe d'améliorations....10
Conclusion....10
RETEX....10
Cuney Alexandre....10
Guyenot-Cosio Amber....10

Introduction

Ce projet a été réalisé dans le cadre de l'UE AP4A. Le but de ce projet est de simuler une environnement IoT (Internet of Things) à l'aide du langage C++ 17. Il nous a été demandé de concevoir une architecture avec des éléments et comportements imposés, tout en y ajoutant des apports personnels. Le but est de simuler un serveur sur lequel est connecté plusieurs types de capteurs effectuant plusieurs mesures, et les enregistrer des fichiers de logs.

Note sur l'IA

Le but de nos études et d'apprendre à faire les choses par nous même, et non pas d'entraîner une IA dans un pays étranger sans RGPD. Ainsi à l'exception du readme, rien n'a été produit par une IA. Notre utilisation à été de manière corrective afin de corriger les erreurs de types, d'appel (référence, pointeurs…) et pour le débogage. En effet les modèles, mêmes grand public, permettent de donner une interprétation aux messages d'erreurs des compilateurs. Le reste à été fait par nos soins, après une démarche réflective des gribouillis sur papier. De ce fait l'intégralité de ce rapport, de la logique son sans IA. Certaines fonctions ont elles bien été corrigées (et seulement corrigées) grâce à des LLMs.

Conception

Structure du projet

Ce projet a été réalisé en c++ et compilé à l'aide de g++ (mais d'autres compilateurs sont également utilisables). Afin de garder un projet ordonné nous avons séparé les différents types de fichiers dans une arborescence claire. Le dossier src (source), contient l'ensemble des fichiers sources (.cpp) afin de les séparer des données traitées. Le dossier headers, contient l'ensemble des fichiers de déclarations des classes C++ (.hpp), ainsi lorsqu'une source à besoin d'appeler une classe il va la chercher de ce fichier. Le dossier data contient l'ensemble des exportations de données dans les fichiers logs, elles sont donc séparées du code en lui-même. Le dossier config contient l'enregistrement des objets utilisés par le programme (Serveur, Sensors, Room) afin de pouvoir les conserver entre les sessions sans les codés en durs. Enfin le dossier build contient le résultat de la compilation, le fichier a exécuter. Nous arrivons donc à l'arborescence suivante :

Racine

```
├── CMakeLists.txt
├── headers/
├── src/
├── bin/
├── config/
└── data/
```

Class et objets

Afin de fonctionner, le logiciel a bien évidemment besoin des classes Server, Scheduler et Sensor, qui sont demandées dans le sujet. De plus, afin d'apporter un intérêt supplémentaire au logiciel, nous avons décidé d'ajouter la classe Ipv4 permettant d'identifier les serveurs et capteurs à l'aide d'une adresse en ipv4 pouvant être attribuée automatiquement par le scheduler (une simulation de DHCP). Par ailleurs, nous avons également ajouté la classe Room, qui simule une salle dans laquelle est installés des capteurs, avec des mesures physiques définies (et non plus aléatoire). Par ailleurs pour garantir un fonctionnement de l'héritage sur les capteurs, nous avons ajouté la classe Data, permettant de gérer différentes type de données ainsi que l'unité standard associée.

Enfin pour sauvegarder et charger les appareils et salles enregistrées on utilise la classe ConfigLoader.

Server

La classe Server simule le serveur. Il est composé d'un IP qui lui sert d'identificateur, un nom, d'un port réseau et d'une version.

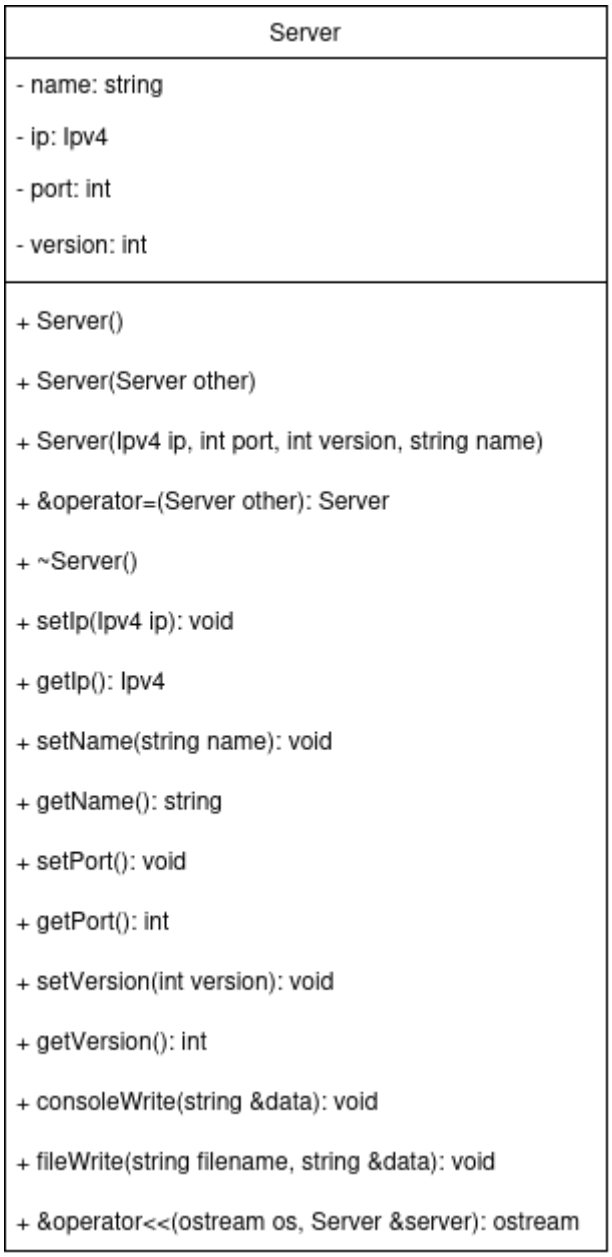


Diagramme UML de Server

Sensor

Les capteurs sont définis par des classes spécifiques telles Thermometer, HumiditySensor, PressureSensor et ParticlesSensor. Ces classes sont toutes des classes fille de la classe à patron Sensor<T>. Le patron de classe permet à chacune des classe fille de redéfinir le type de données mémoire manipuler à l'aide de la classe Data. Seulement afin de manipuler les capteurs depuis ordonnancier (Scheduler), nous avons implémenté la classe mère Isensor, servant d'interface sans patron, qui définit l'ensemble des classes et méthodes utiles aux capteurs. Cette classe est héritée par Sensor qui est héritée par les différentes classes de capteurs.

Ainsi les méthodes update et execute sont déclarées dans Isensor, mais tout ce qui utilise Data (getData(), measurement) sont déclarées à partir de Sensor. La fonction execute génère une nouvelle donnée à partir de la pièce où les capteurs sont installés, puis pioche une valeur comprise dans un intervalle définie à partir de la réelle mesure, et du taux de confiance de l'appareil (accuracy) pour simuler un capteurs plus ou moins fiable. La fonction update est appelée par l'ordonnancier.

L'attribut Data lui permet de définir le type et l'unité de la mesure. Il est redéfinie à chaque classes filles.

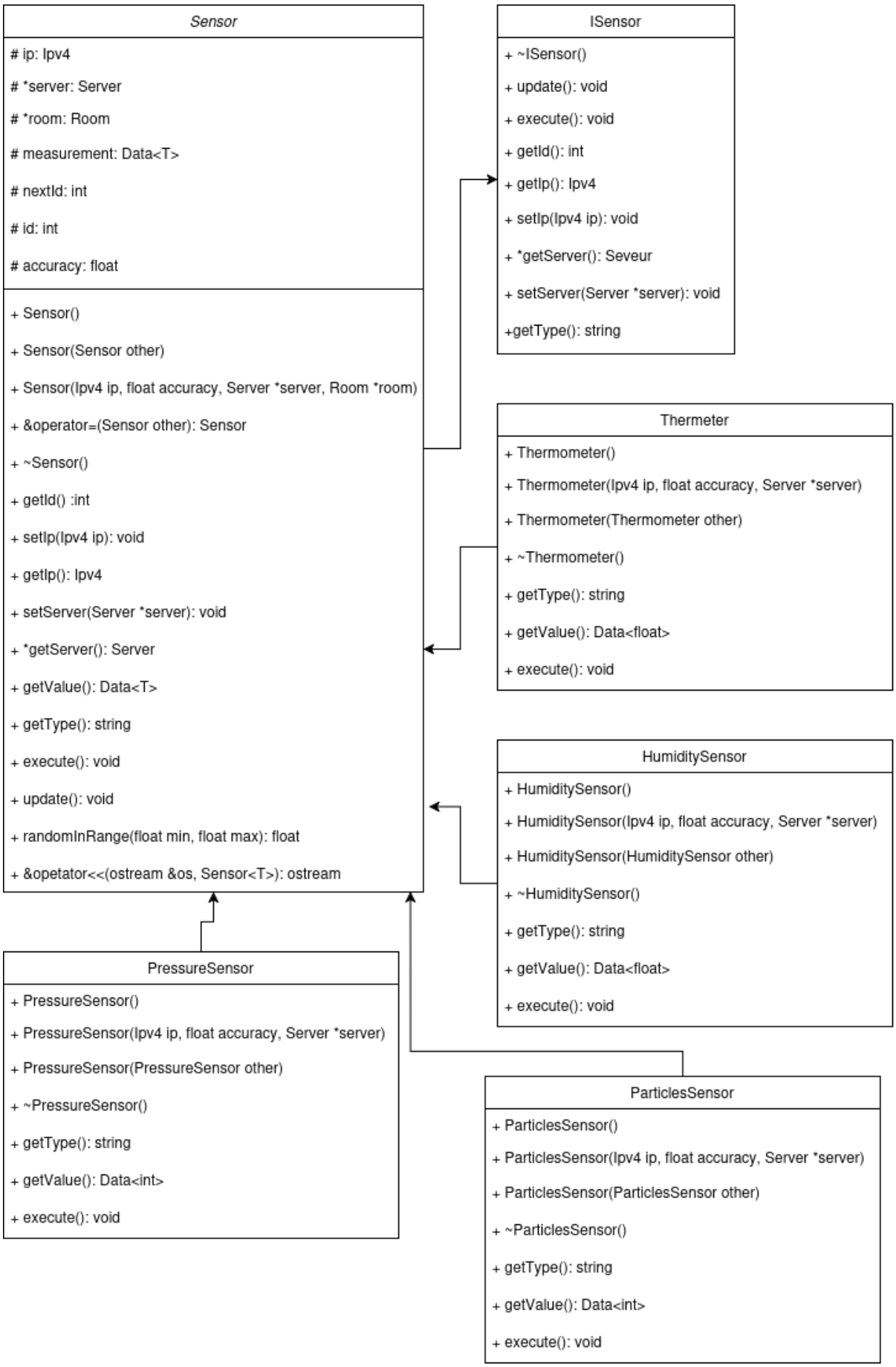


Diagramme UML de Sensor

Scheduler

L’ordonnancier est défini par la classe Scheduler. Il contient une liste de tout les capteurs utilisés, contenu dans un vecteur issue de la STL. Il contient aussi la procédure simulation, qui exécute la simulation.

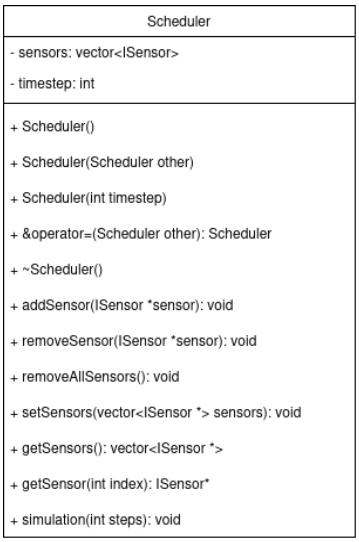


Diagramme UML de Scheduler

Room

Les pièces physique sont définie par la classe Room, elle contient l’ensemble des données physiques mesurable par les différents capteurs, une identifiant unique ainsi qu’un nom défini par l’utilisateur.

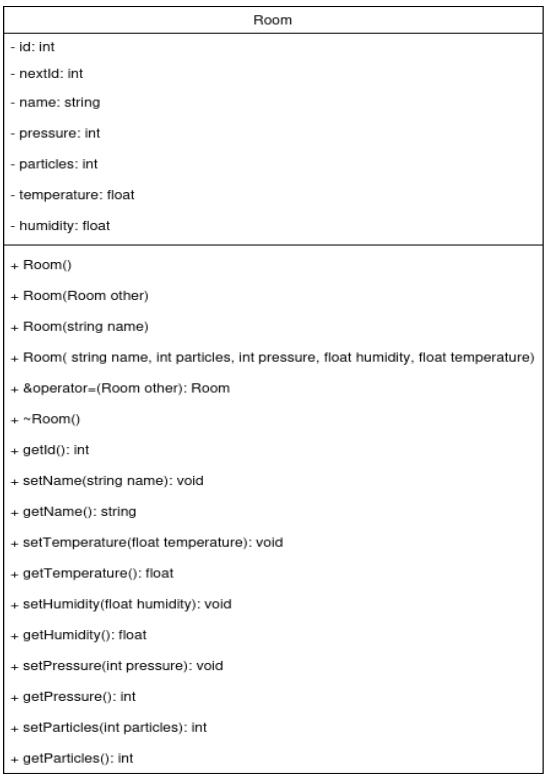


Diagramme UML de Room

Data

La classe Data permet de manipuler des données de n'importe quel type. Ainsi que de gérer les unités standard. C'est une classe avec patron qui est utilisée par Sensor, et est à l'origine de la nécessité de l'interface Isensor.

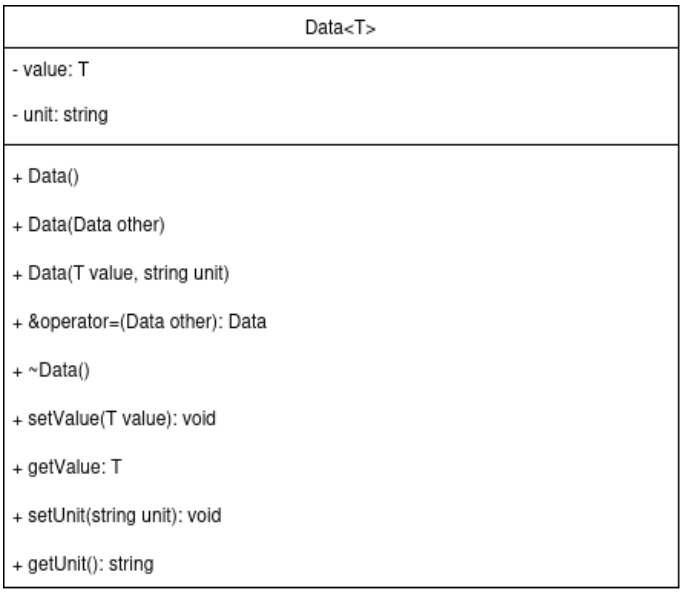


Diagramme UML de Data

Ipv4

La classe Ipv4 sert à simuler une adresse ipv4 composé donc de quatre octets, sur lesquels sont codé l'adresse d'un périphérique.

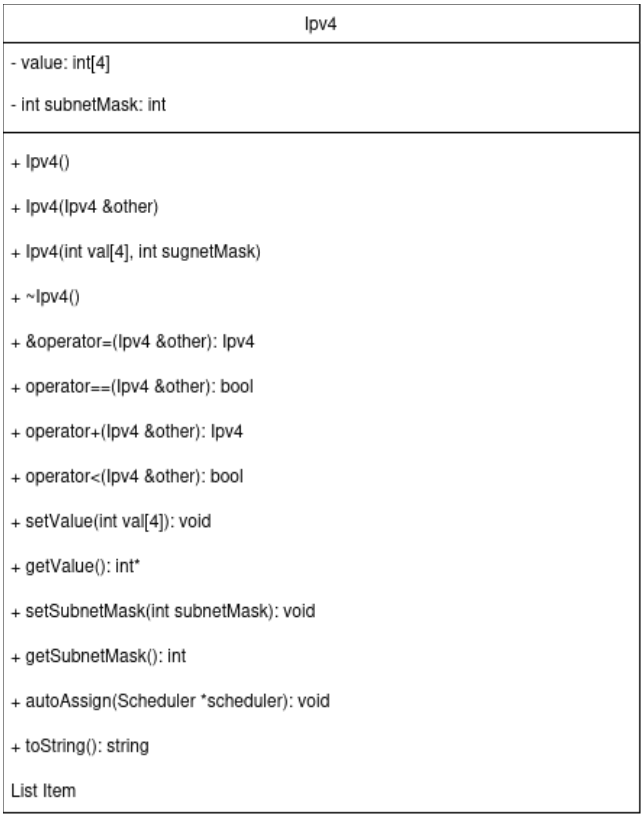


Diagramme UML de Ipv4

ConfigLoader

ConfigLoader ne contient que des méthodes, ces méthodes permettent de charger les serveurs, capteurs et salles depuis le dossier config.

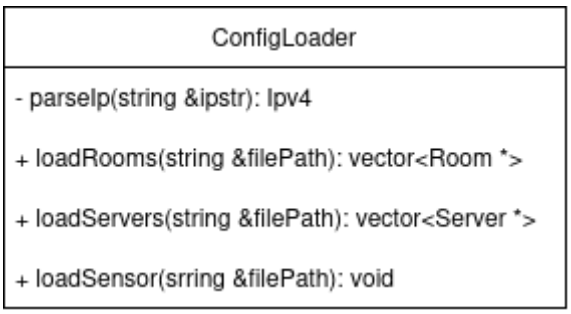


Diagramme UML de ConfigLoader

Fonctionnement

Le fonctionnement général du logiciel est plutôt simple, l'ordonnancier demande des mesures à tous ces capteurs, qui vont à leur tour lire la valeur dans les salles, avec plus ou moins d'exactitude. Les capteurs et serveurs possèdent des IP et les capteurs un des Data.

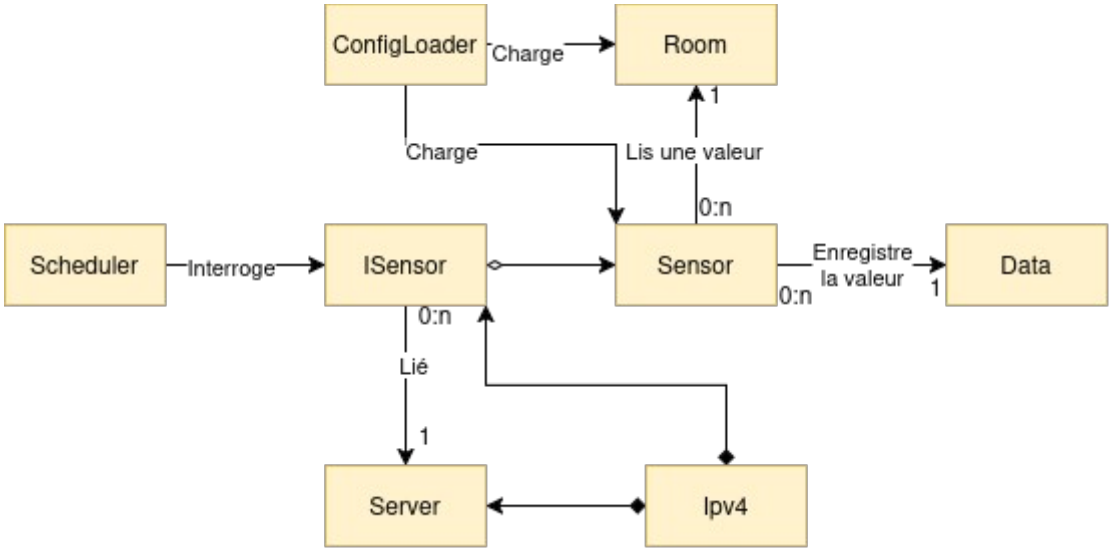


Diagramme UML du focntionnement du logiciel

Axe d’améliorations

Bien que le logiciel répond actuellement au sujet et même plus, il est toujours possible de l’améliorer. En effet actuellement le serveur reste passif et les données ne sont pas traitées. On pourrait imaginer que le serveur analyse les données lue et tire des conclusion en conséquences (détections d’anomalie, incendies…). De plus actuellement le seul moyen d’ajouter des périphériques et des salles et de modifier les fichiers de configuration, c’est bien mieux que de le coder en dur, mais on pourrait imaginer des commandes permettant de poser des questions à l’utilisateur pour que le programme ajoute lui même ces entrées dans les fichiers.

Un autre ajout qui pourrait modifier l’expérience utilisateur est l’ajout d’une interface graphique permettant de d’utiliser le logiciels sans entrer dans les fichiers de configuration ou utiliser de commandes.

Conclusion

Nous pouvons conclure que le logiciel est fonctionnel, et nous permet d’utiliser toutes les notions abordées dans l’UE AP4A. Le logiciel est toujours améliorable, en terme de qualité logiciel, d’UX, d’UI et d’optimisation.

RETEX

Cuney Alexandre

J’ai l’habitude de la programmation orientée objet, mais avec Java et PHP. Cela n’as pas été très long de m’adapter au C++, bien que je lui trouve de nombreux défauts par rapport aux langages cités plus tôt. Cependant j’ai pu explorer d’autres points que je n’avais pas encore pratiquer tel que les classes et fonctions avec patrons. De plus la gestion des pointeurs et de références est une particularité que je n’utilise pas habituellement avec Java. Par ailleurs je n’est pas l’habitude de ce type de projet , dans le sens de la logique métier et l’utilisation.

Guyenot-Cosio Amber

La réalisation de ce projet de simulateur IoT m'a a permis de mettre concrètement en pratique l'ensemble des concepts de la programmation orientée objet, en passant d'une architecture de base (forme canonique de Coplien, gestion centralisée du serveur et de l'ordonnanceur) à une structure générique complète exploitant l'héritage, l'encapsulation et

le polymorphisme. L'intégration d'un module de chargement par fichiers de configuration (ConfigLoader) et de l'adressage IP dynamique a apporté une grande flexibilité à l'écosystème. La principale difficulté a résidé dans la gestion rigoureuse de la mémoire et des pointeurs partagés entre le serveur, les pièces et les capteurs pour éviter les fuites et erreurs d'allocation (free(): invalid pointer). Enfin, la mise en place d'une chaîne de compilation multi-plateforme propre via CMake et l'organisation modulaire du code ont constitué un véritable atout pour le travail en binôme.